Plano de Implementação — Autenticação Funcional (Login & Cadastro)

Este plano define a criação e integração ponta a ponta do sistema de **Cadastro e Login 100% funcional** para o **SEHA (Engenharia de Hábitos)**, atendendo rigorosamente aos requisitos **RF01**, **RF02**, **RNF01**, **RNF02**, **RNF03** e **RNF04** da especificação.

1. Arquitetura da Solução

Conforme a especificação:

- **Back-end:** Python com **FastAPI**, validação com **Pydantic**, ORM **SQLAlchemy** e suporte a **Supabase Auth**.
- **Front-end: Next.js**, consumindo a API do FastAPI via context de autenticação (`AuthContext`) e gerenciando o estado de sessão de forma segura.

```
[ Navegador / Next.js ]
   ├── /cadastro ──(POST /api/auth/register)──► [ FastAPI Backend ]
   ├── /login ─────(POST /api/auth/login)─────►   ├── Validação Pydantic (RNF02)
   ├── AuthContext ──(GET /api/auth/me)───────►   ├── Hash / Supabase Auth (RNF01)
   └── Rotas Protegidas (/dashboard, etc.)        └── Tabela USUARIOS (SQLAlchemy)
```

2. Etapas de Implementação

Etapa 1: Estrutura do Back-end FastAPI (`backend/`)

1. **Configuração do ambiente e dependências:**
    - Criar pasta `backend/` com `requirements.txt` :
        - `fastapi` , `uvicorn[standard]` , `pydantic[email]` , `sqlalchemy` , `python-jose[cryptography]` , `passlib[bcrypt]` , `python-dotenv` , `supabase` .
2. **Configuração de Ambiente e Conexão (`backend/app/core/`):**
    - `config.py` : leitura de variáveis de ambiente (`SECRET_KEY` , `SUPABASE_URL` , `SUPABASE_KEY` , `DATABASE_URL`).

- Suporte híbrido inteligente:
  - Se credenciais do Supabase forem fornecidas no `.env`, integra diretamente com o Supabase Auth API / PostgreSQL do Supabase.
  - Fallback local seguro (JWT + bcrypt + SQLite/PostgreSQL) caso o usuário queira rodar localmente antes de configurar as chaves do Supabase, garantindo funcionamento imediato sem travas.

3. **Modelos de Banco de Dados (`backend/app/models/user.py`):**
   - Tabela `usuarios`:
     - `id`: string (UUID)
     - `nome`: string
     - `email`: string (único)
     - `senha_hash`: string (protegida por hash bcrypt, nunca texto puro - RNF01)
     - `criado_em`: datetime
4. **Schemas Pydantic (`backend/app/schemas/auth.py` - RNF02):**
   - `UserRegister`: validação de nome obrigatório, formato de e-mail e senha mínima de 6 caracteres.
   - `UserLogin`: validação de e-mail e senha.
   - `UserResponse`: retorno seguro (id, nome, email, criado_em - sem senha).
   - `TokenResponse`: token JWT e payload do usuário.
5. **Rotas de Autenticação (`backend/app/api/auth.py`):**
   - `POST /api/auth/register`: cadastra usuário, valida unicidade de e-mail, gera hash seguro da senha e emite token de acesso.
   - `POST /api/auth/login`: valida e-mail e senha, rejeita credenciais incorretas e emite token de acesso.
   - `GET /api/auth/me`: decodifica o token no cabeçalho `Authorization: Bearer <token>` e retorna os dados do usuário autenticado.

Etapa 2: Integração no Front-end Next.js (`frontend/`)

1. **Serviço de Comunicação (`src/services/api.ts`):**
   - Configuração de cliente HTTP apontando para `NEXT_PUBLIC_API_URL` (padrão `http://localhost:8000`).
   - Interceptor automático para injetar o header `Authorization: Bearer <token>`.

2. **Contexto de Autenticação (`src/contexts/AuthContext.tsx`):**
    - Gerencia o estado global de autenticação: `user`, `token`, `isLoading`, `login`, `register`, `logout`.
    - Persistência de token segura no `localStorage` do navegador para manter a sessão ao recarregar a página.
    - Verificação automática da sessão ativa na inicialização (`/api/auth/me`).
3. **Atualização da Tela de Cadastro (`src/app/cadastro/page.tsx`):**
    - Conectar o formulário à função `register()` real.
    - Tratar e exibir erros retornados pelo back-end (ex: "E-mail já cadastrado", "Senha fraca").
    - Redirecionar automaticamente para o `/dashboard` após cadastro bem-sucedido.
4. **Atualização da Tela de Login (`src/app/login/page.tsx`):**
    - Conectar o formulário à função `login()` real.
    - Tratar erros de credenciais inválidas.
    - Redirecionar para `/dashboard`.
5. **Atualização do `Navbar.tsx` e Telas Protegidas:**
    - Exibir o nome real do usuário autenticado no topo.
    - Ação de "Sair" (Logout) funcional, limpando a sessão e redirecionando para a tela inicial.
    - Adicionar guarda no `/dashboard` para redirecionar usuários não autenticados para `/login`.

3. Plano de Verificação

1. **Teste de Cadastro:**
    - Tentar cadastrar com e-mail inválido ou senha curta (validar retorno do Pydantic).
    - Cadastrar usuário real `teste@exemplo.com`.
    - Verificar persistência segura no banco de dados com senha em hash.
2. **Teste de Login:**
    - Tentar logar com senha errada (esperar mensagem clara de erro).
    - Logar com a senha correta (esperar sucesso, redirecionamento para o dashboard e exibição do nome do usuário).
3. **Teste de Sessão & Logout:**

- Recarregar a página no Dashboard e verificar se o login se mantém.
- Clicar em "Sair" e verificar se o estado volta para visitante e o acesso a dados privados é bloqueado.